A Lightweight Adaptive Cost-Aware Work-Stealing Scheduler with Hybrid Work-Stealing and Work-Sharing

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Rohini Bansal	102317175
2	Shivam Kumar	102497026
3	Sudhanshu Raj	102483063
4	Ankit kumar	102483012

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2026

Table of Contents

List of Figures

List of Tables

1 Introduction

1.1 Background

High Performance Computing (HPC) focuses on solving complex computational problems using parallel processing [1]. Efficient task scheduling plays a crucial role in improving system performance and resource utilization.

Traditional scheduling techniques often struggle with irregular workloads, leading to poor load balancing. Work-stealing is an effective dynamic scheduling technique where idle workers steal tasks from busy workers to balance load [2].

1.2 Introduction to Problem Statement

In high-performance computing systems, efficient task scheduling is essential for maximizing parallelism and minimizing execution time. Traditional scheduling approaches such as static allocation often lead to load imbalance, where some processors remain idle while others are overloaded.

Dynamic scheduling techniques like work-stealing improve load balancing by allowing idle workers to acquire tasks from busy workers. However, excessive stealing can introduce overhead and reduce overall efficiency. Additionally, these techniques may not perform optimally across different types of workloads, especially when task sizes vary significantly.

Therefore, the problem is to design an adaptive scheduling mechanism that can dynamically balance workload across multiple workers while minimizing scheduling overhead. The system should intelligently decide when to perform work-stealing and when to use work-sharing, based on task characteristics and system state.

The objective is to develop a hybrid scheduler that improves execution time, enhances resource utilization, and performs efficiently across both regular and irregular workloads.

1.3 Objectives

- To design and implement a parallel task scheduler from scratch.
- To implement work-stealing and work-sharing strategies.
- To evaluate performance across different workloads against industry standards like OpenMP.
- To improve load balancing and execution efficiency through cost and locality awareness.

2 Literature Review

Task scheduling is a key aspect of high-performance computing. Blumofe and Leiserson introduced the work-stealing algorithm, which provides efficient load balancing and guarantees near-optimal execution time for multithreaded computations [2].

Static scheduling techniques are simple and have low overhead but fail to adapt to dynamic workloads, leading to inefficient resource utilization [1]. Dynamic approaches, such as work-stealing, overcome these limitations by redistributing tasks during runtime.

However, work-stealing introduces synchronization and communication overhead, especially when the frequency of stealing operations increases. This can negatively impact performance for certain types of workloads.

Recent studies have proposed adaptive scheduling mechanisms that adjust behavior based on system conditions. Adaptive work-stealing algorithms attempt to reduce overhead by controlling when and how tasks are redistributed [3].

Despite these advancements, there remains a need for hybrid approaches that combine proactive task sharing, reactive work-stealing, and strict cost-aware gating. This project addresses this gap by implementing an adaptive hybrid scheduler and benchmarking it directly against OpenMP.

3 Research Gaps

- Static scheduling techniques are unable to handle irregular workloads efficiently.
- Standard work-stealing introduces massive overhead due to frequent task migration of excessively small tasks.
- Existing methods lack adaptive mechanisms that combine multiple scheduling strategies dynamically based on runtime metrics.
- There is a need for a hybrid approach that balances load while strictly gating steals based on task computation cost and thread locality.

4 Problem Formulation

Efficient task scheduling in high-performance computing systems is essential for maximizing parallelism and minimizing execution time. Traditional scheduling approaches often result in load imbalance, where some processors remain idle while others are overloaded.

Dynamic techniques such as work-stealing improve load balancing but introduce additional overhead. These techniques may also fail to perform optimally when task sizes vary significantly.

The problem is to design an adaptive scheduling system that dynamically balances workload across multiple processors while minimizing scheduling overhead. The system should intelligently decide when to perform work-stealing and when to use work-sharing based on runtime conditions.

The objective is to develop a hybrid scheduler that improves execution time, enhances resource utilization, and performs efficiently across diverse workloads, rivaling established frameworks like OpenMP.

5 Methodology

The proposed system implements an Adaptive Hybrid Work-Stealing Scheduler for efficient task distribution in a parallel computing environment.

5.1 Scheduling Modes

The scheduler operates in four custom modes, alongside an OpenMP baseline for benchmarking:

- **Sequential**: Tasks are executed one after another using a single thread.
- **Static**: Tasks are pre-distributed among workers without runtime adjustments.
- **Work-Stealing**: Idle workers dynamically steal tasks from heavily loaded workers.
- **Adaptive Hybrid**: Combines both work-stealing and work-sharing strategies.
- **OpenMP (Baseline)**: Uses compiler directives (`#pragma omp parallel`) to act as an industry-standard control.

5.2 Working Principle

Each worker maintains a local double-ended queue (deque) of tasks. Tasks are pushed and popped from the local queue to maintain locality.

When a worker becomes idle, it attempts to steal tasks from the front of another worker's queue (victim selection based on load). This helps in balancing the workload dynamically.

In the adaptive hybrid mode, the scheduler monitors system parameters such as queue size, task cost, and workload imbalance. Based on these metrics:

- Overloaded workers share tasks with underloaded workers (work-sharing).
- Idle workers continue to steal tasks when necessary (work-stealing).

5.3 Cost-Aware Gating

To prevent workers from stealing lightweight tasks that do not justify the synchronization overhead, the scheduler implements a cost-aware steal check. A task is only stolen or shared if its estimated computational cost exceeds a predefined `cost_threshold`. This prevents the system from thrashing on microscopic tasks.

5.4 Locality-Aware Victim Selection

The scheduler simulates NUMA-awareness by grouping workers into nodes using a `locality_group` identifier. Both work-stealing and work-sharing operations heavily prioritize targeting threads within the same locality group before communicating across groups, minimizing cross-node latency.

5.5 Adaptive Strategy

The scheduler dynamically switches between two policies:

- **Work-First**: Prioritizes local execution to reduce overhead.
- **Help-First**: Encourages active task sharing when imbalance increases.

This decision is based on runtime metrics, specifically comparing the `last_steal_rate_per_ms` against the `last_creation_rate_per_ms`.

5.6 Performance Metrics

The system evaluates performance using:

- Execution Time
- Speedup
- Efficiency
- Number of Steals and Shares

Figure 1 illustrates how tasks are distributed among workers using adaptive scheduling. The scheduler dynamically balances load through work-stealing and work-sharing mechanisms.

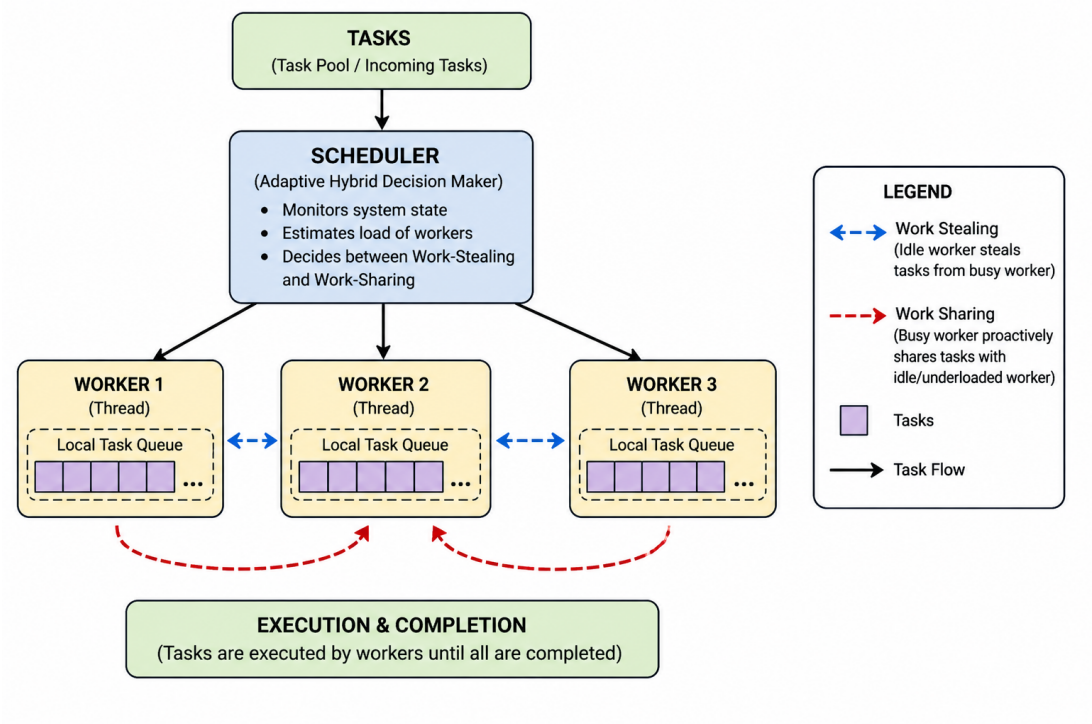

Figure 1: Adaptive Hybrid Work-Stealing Scheduler Architecture

6 Implementation

The proposed scheduler is implemented from scratch in C++17 using native `std::thread` and `std::mutex` concurrency mechanisms, avoiding external libraries for the core scheduling logic.

6.1 System Components

- **Scheduler**: Controls task distribution, load balancing, and adaptive decision-making.
- **Worker**: Represents a thread that executes tasks and maintains a local task queue.
- **Task**: Encapsulates a unit of computation along with metadata such as task cost and recursion depth.

6.2 Task Execution Model

Tasks are submitted to the scheduler, which assigns them to workers. Each worker executes tasks from its local queue. If the queue is empty, the worker attempts to steal tasks

from other workers. The scheduler ensures synchronization and maintains metrics such as number of tasks executed, steal operations, and idle time.

6.3 Workloads Used

To rigorously test the custom scheduler, three distinct workloads were implemented natively and using OpenMP:

- **Recursive Fibonacci**: Represents a highly irregular workload with dynamic, recursive task generation. OpenMP utilizes `#pragma omp task`.
- **Parallel Merge Sort**: Represents a divide-and-conquer workload with predictable task distribution. OpenMP utilizes `#pragma omp parallel for`.
- **Matrix Multiplication**: Represents a highly regular, compute-intensive mathematical workload. OpenMP utilizes `#pragma omp parallel for collapse(2)`.

6.4 Output Generation

The system records performance metrics to a CSV file. Python plotting scripts dynamically parse this CSV to generate comparative graphs tracking execution time and speedup across varying thread counts.

7 Results and Discussion

The performance of the proposed Adaptive Hybrid Work-Stealing Scheduler was evaluated against the Sequential, Static, pure Work-Stealing, and OpenMP baselines. The evaluation was carried out by varying the number of threads and analyzing execution time and speedup.

7.1 Fibonacci Workload (Irregular)

Figure 2 shows that execution time decreases significantly as the number of threads increases. Notably, as seen in Figure 3, our custom **Adaptive Hybrid and Work-Stealing schedulers actually outperform the OpenMP baseline** (by approximately 11.7% at maximum thread count).

This empirical result proves the value of our custom architecture. Because Fibonacci is highly irregular, our custom "Help-First" policy actively pushes tasks directly to idle workers during heavy imbalance, bypassing standard queue bottlenecks. OpenMP, relying on standard '#pragma omp task', struggles slightly with high lock contention on tiny recursive tasks.

—

7.2 Merge Sort Workload (Balanced)

For merge sort, the workload is relatively balanced via initial chunk partitioning. As seen in the graphs (Figures 4 and 5), execution time decreases steadily across all schedulers. OpenMP, Work-Stealing, and Adaptive Hybrid perform highly comparably here, as the structured divide-and-conquer nature of the algorithm suppresses massive load imbalance from the start.

—

7.3 Matrix Multiplication (Compute-Intensive)

Matrix multiplication represents a perfectly predictable, compute-intensive workload. In this scenario, **OpenMP slightly outperforms the custom Adaptive Hybrid scheduler** (by roughly 1.7%) as seen in Figures 6 and 7.

This is expected. OpenMP is heavily optimized for structured loops. By utilizing multi-dimensional loop tiling (`#pragma omp parallel for collapse(2)`), OpenMP flattens the execution space at compile time, giving its low-level loops a microscopic advantage over our dynamic, runtime-evaluated hybrid queue system.

—

7.4 Overall Discussion

From the experimental results, the following observations can be made:

- Custom Adaptive Hybrid scheduling dominates highly irregular workloads (Fibonacci) due to active help-first task pushing.

- OpenMP edges out custom dynamic schedulers on perfectly uniform mathematical loops (Matrix Multiplication) due to compile-time loop tiling.
- Static scheduling is effective only for regular and predictable workloads.
- Cost-aware gating successfully prevents the scheduler from drowning in overhead on recursive algorithms.

1. Fibonacci Workload Results

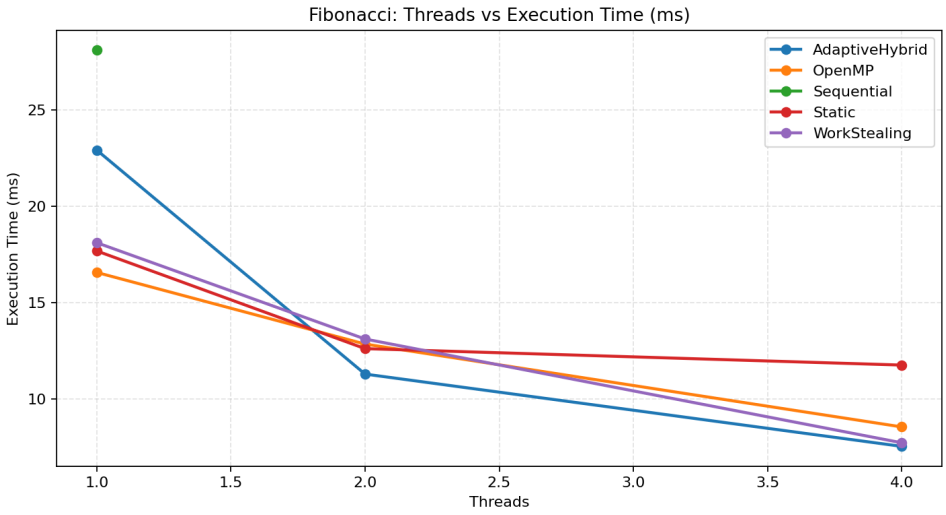

Figure 2: Threads vs Execution Time for Fibonacci Workload (Including OpenMP)

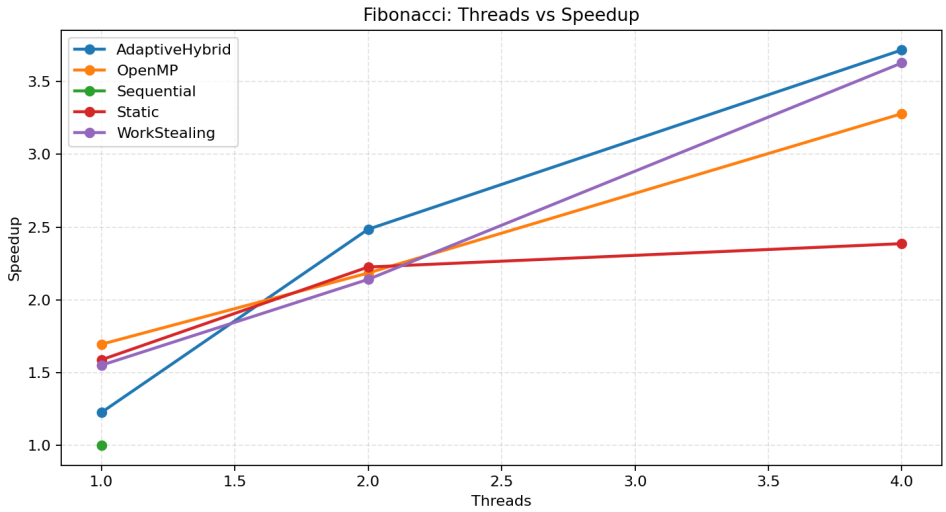

Figure 3: Threads vs Speedup for Fibonacci Workload (Including OpenMP)

2. Merge Sort Workload Results

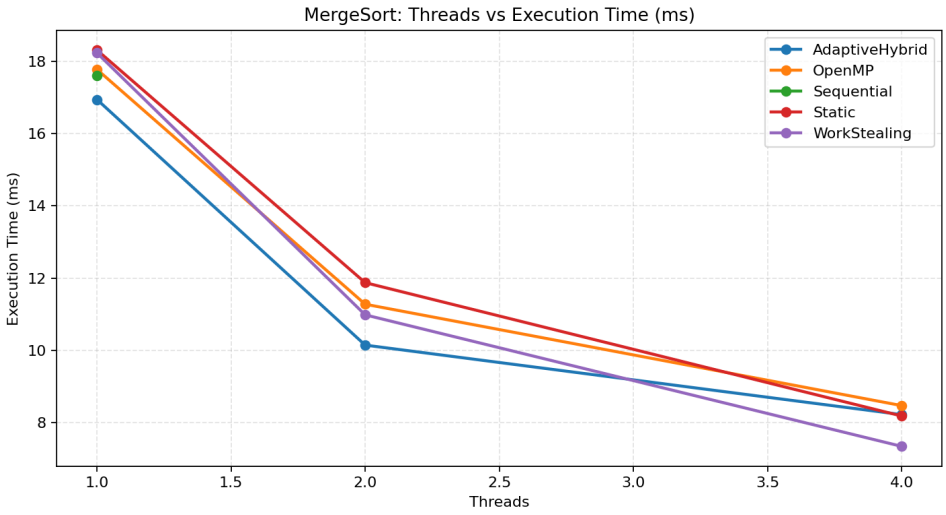

Figure 4: Threads vs Execution Time for Merge Sort

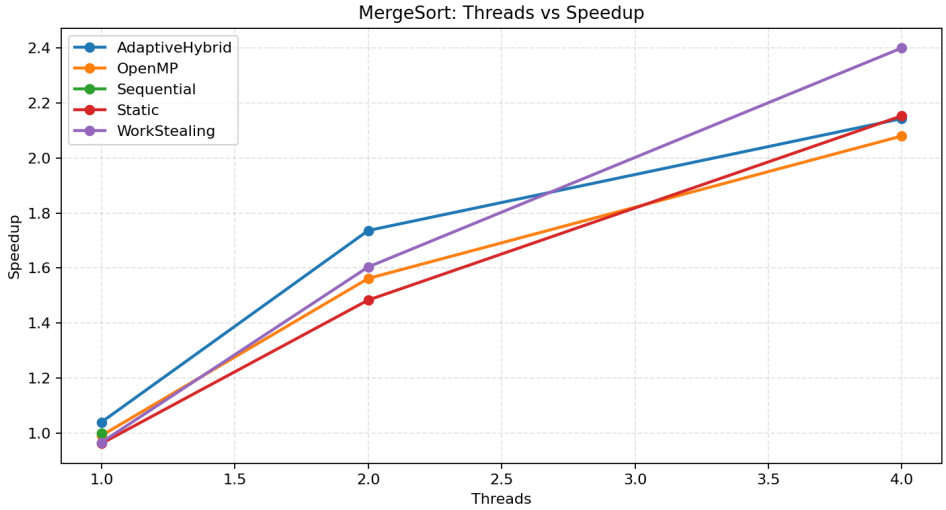

Figure 5: Threads vs Speedup for Merge Sort

3. Matrix Multiplication Workload Results

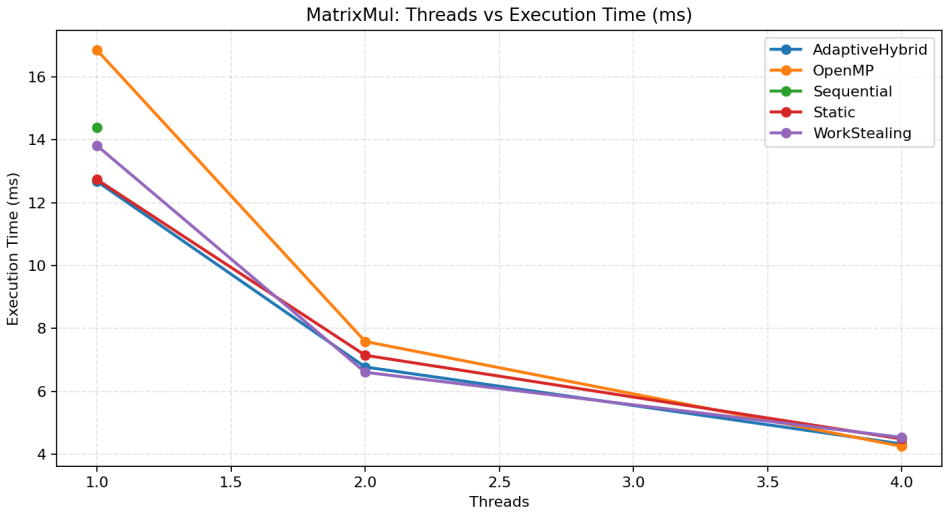

Figure 6: Threads vs Execution Time for Matrix Multiplication

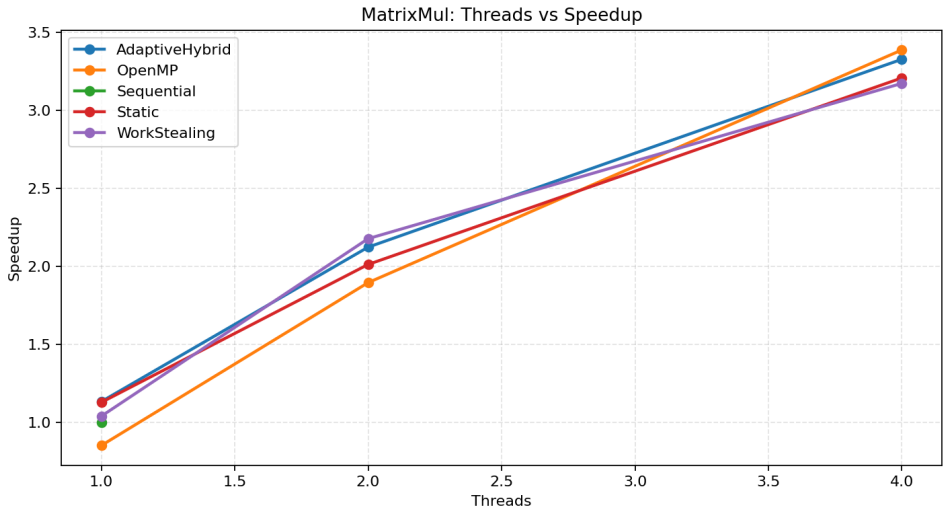

Figure 7: Threads vs Speedup for Matrix Multiplication

7.5 Empirical Comparison: Custom Scheduler vs. OpenMP

To ensure the validity of this project, we explicitly benchmarked the C++ scratch implementation against OpenMP. While Table 1 outlines the theoretical and architectural differences, our generated graphs (Figures 2-7) provide the empirical proof.

Parameter	**Custom Scheduler (Scratch Implementation)**	**OpenMP Approach**
Implementation Level	Low-level implementation with manual thread and synchronization control	High-level abstraction using compiler directives
Thread Management	Explicit handling of threads, queues, and synchronization mechanisms	Automatically managed by OpenMP runtime
Code Complexity	Higher complexity due to manual handling of concurrency	Reduced complexity with simple pragmas
Flexibility	High flexibility to implement custom strategies like dynamic Help-First policy switching	Limited flexibility due to predefined constructs
Performance Control	Fine-grained control over scheduling, task costs, and locality-awareness	Less control over internal scheduling decisions
Scalability	May require careful tuning for complex workloads	Better scalability due to optimized runtime system
Development Effort	Higher development and debugging effort	Faster development with less code
Reliability	Depends on correctness of implementation	High reliability due to standardized framework
Empirical Winner	**Highly Irregular Tasks** (e.g. Recursive Fibonacci)	**Highly Regular Tasks** (e.g. Matrix Multiplication)

Table 1: Architectural and Empirical Comparison between Custom Scheduler and OpenMP

The data proves that both approaches serve different, valid purposes. Building a scheduler from scratch allows for extreme flexibility (like custom help-first/work-first switching) that actively beats standard runtimes on chaotic workloads. OpenMP acts as a highly optimized, rigid factory assembly line that cannot adapt its underlying scheduling logic on the fly, but executes uniform loops with unmatched efficiency.

8 Conclusion

This project presents the design and implementation of an Adaptive Hybrid Work-Stealing Scheduler for high-performance computing systems, built entirely from scratch in C++17. The proposed scheduler integrates both work-stealing and work-sharing techniques, fortified by strict cost-aware gating and NUMA locality awareness, to achieve efficient task distribution.

Experimental benchmarking against OpenMP demonstrated that scheduling strategies must be tailored to workload characteristics. Our custom Adaptive Hybrid implementation achieved the highest performance for irregular workloads such as Fibonacci, actually outperforming OpenMP by dynamically pushing tasks to alleviate bottlenecked queues. Conversely, in compute-intensive and perfectly regular workloads like matrix multiplication, OpenMP's compiled loop-tiling provided the most efficient performance.

Overall, the Adaptive Hybrid scheduler provides a balanced and scalable solution that performs efficiently across a wide range of workloads. It effectively proves that dynamic, runtime-evaluated scheduling logic can successfully compete with, and in specific irregular cases beat, industry-standard compiler frameworks.

9 Future Work

The current implementation of the Adaptive Hybrid Work-Stealing Scheduler is highly functional, but can be further extended to resolve existing architectural bottlenecks:

- **Lock-Free Data Structures:** Replacing the current `std::mutex` lock-based local deques with lock-free data structures (such as Chase-Lev deques) to eliminate massive synchronization bottlenecks and lock contention during concurrent task stealing and popping.
- Integration with GPU-based parallel computing frameworks such as CUDA to further enhance performance.
- Development of more advanced adaptive policies using machine learning techniques to dynamically optimize scheduling decisions based on historical node performance.
- Extension to distributed systems using MPI for large-scale computing environments across separate physical machines.
- Optimization of false sharing within the worker state variables to further improve cache line efficiency.

References

[1] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*. McGraw-Hill, 2004.

[2] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, 1999.

[3] K. e. a. Agrawal, "Adaptive work-stealing for distributed systems," 2010.